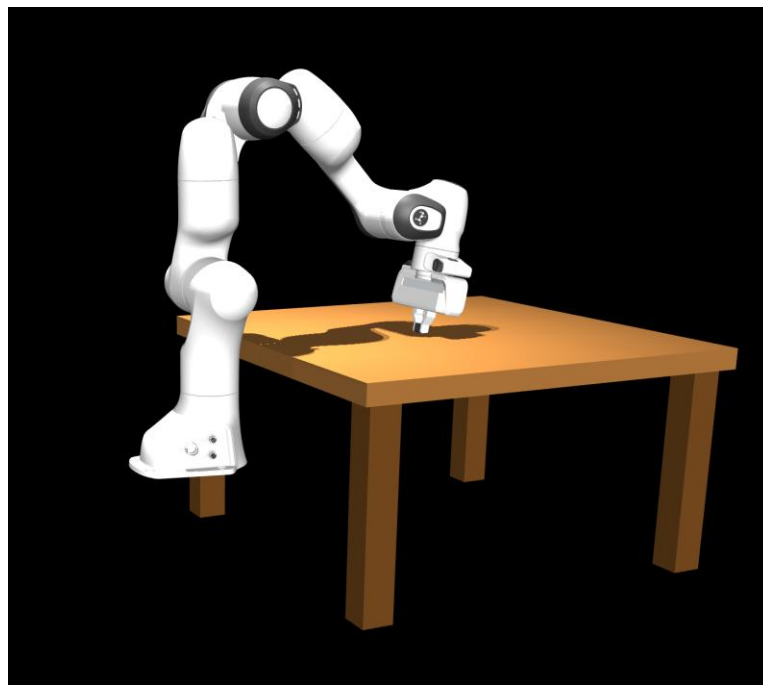


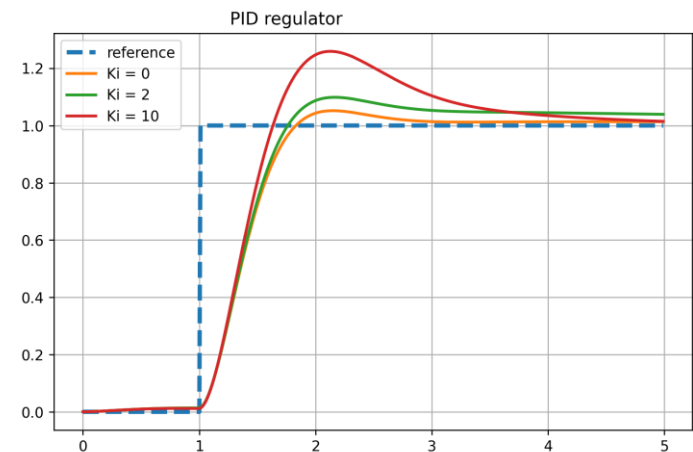
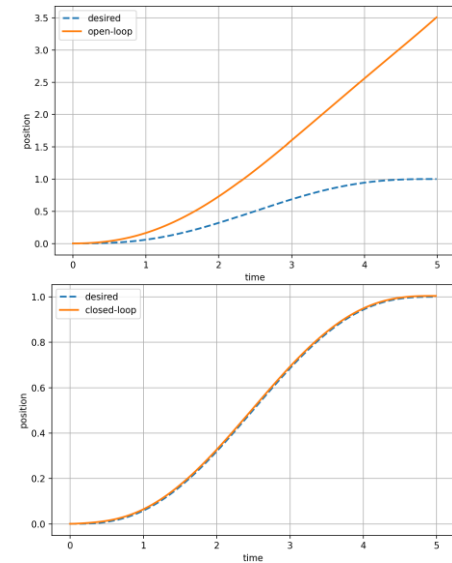
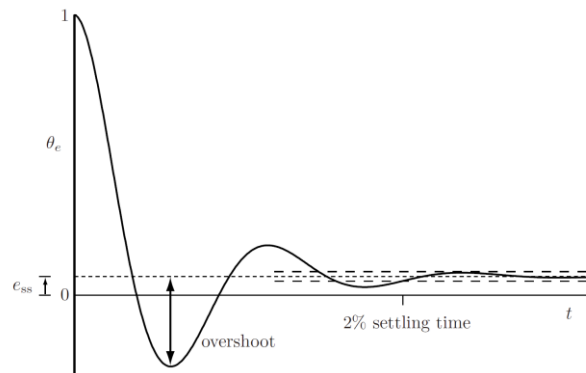
Upravljanje i interakcija kompleksnih robotskih sustava

Tara Knežević,
mag. ing. mech.

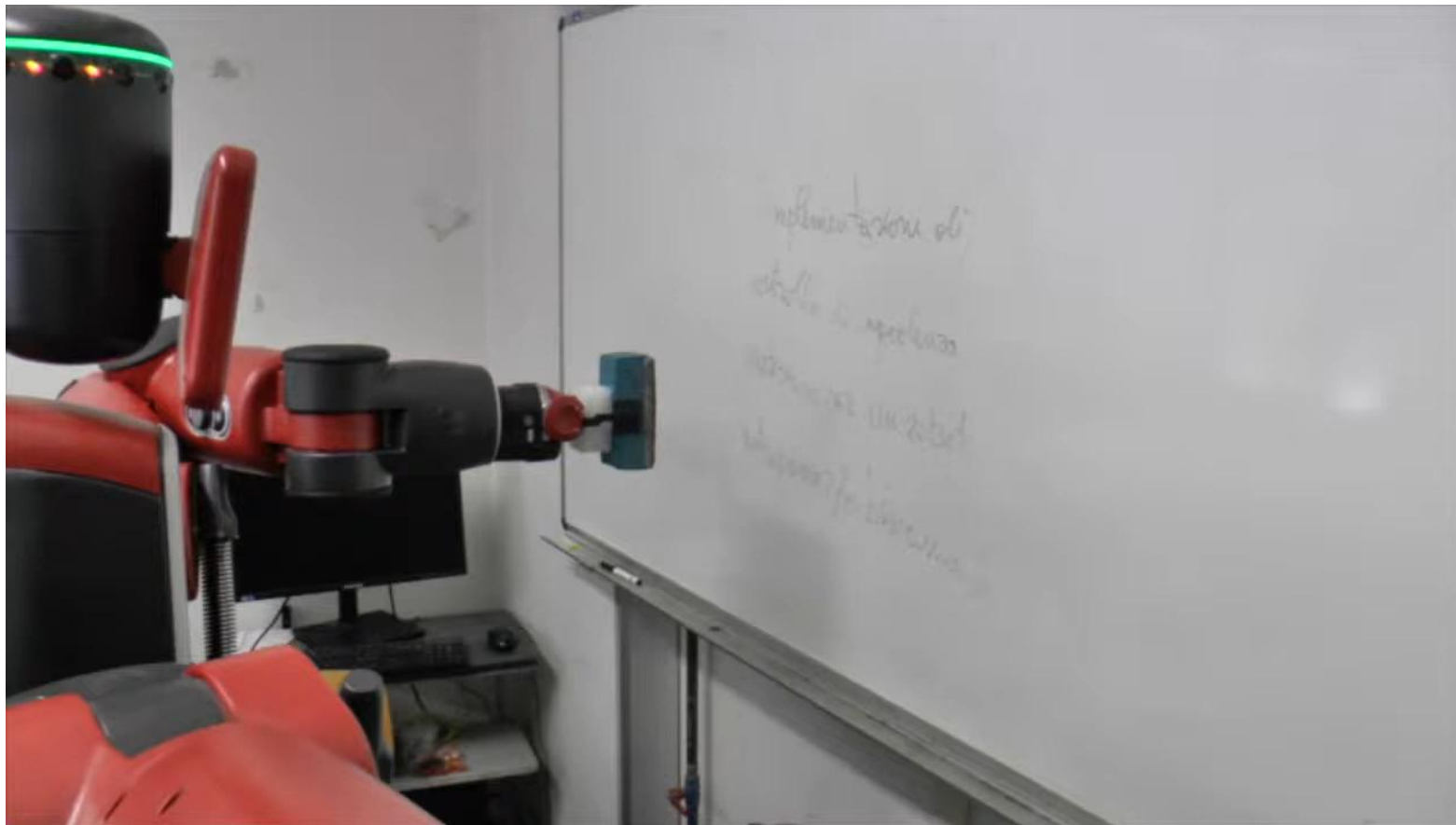


Ponavljjanje

- Do sada smo radili:
 - Regulaciju jednostavnijih sustava
 - PID upravljanje
 - Joint i task space control
- Cilj je bio:
 - Pratiti zadanu trajektoriju
 - Minimizirati grešku



Motivacija



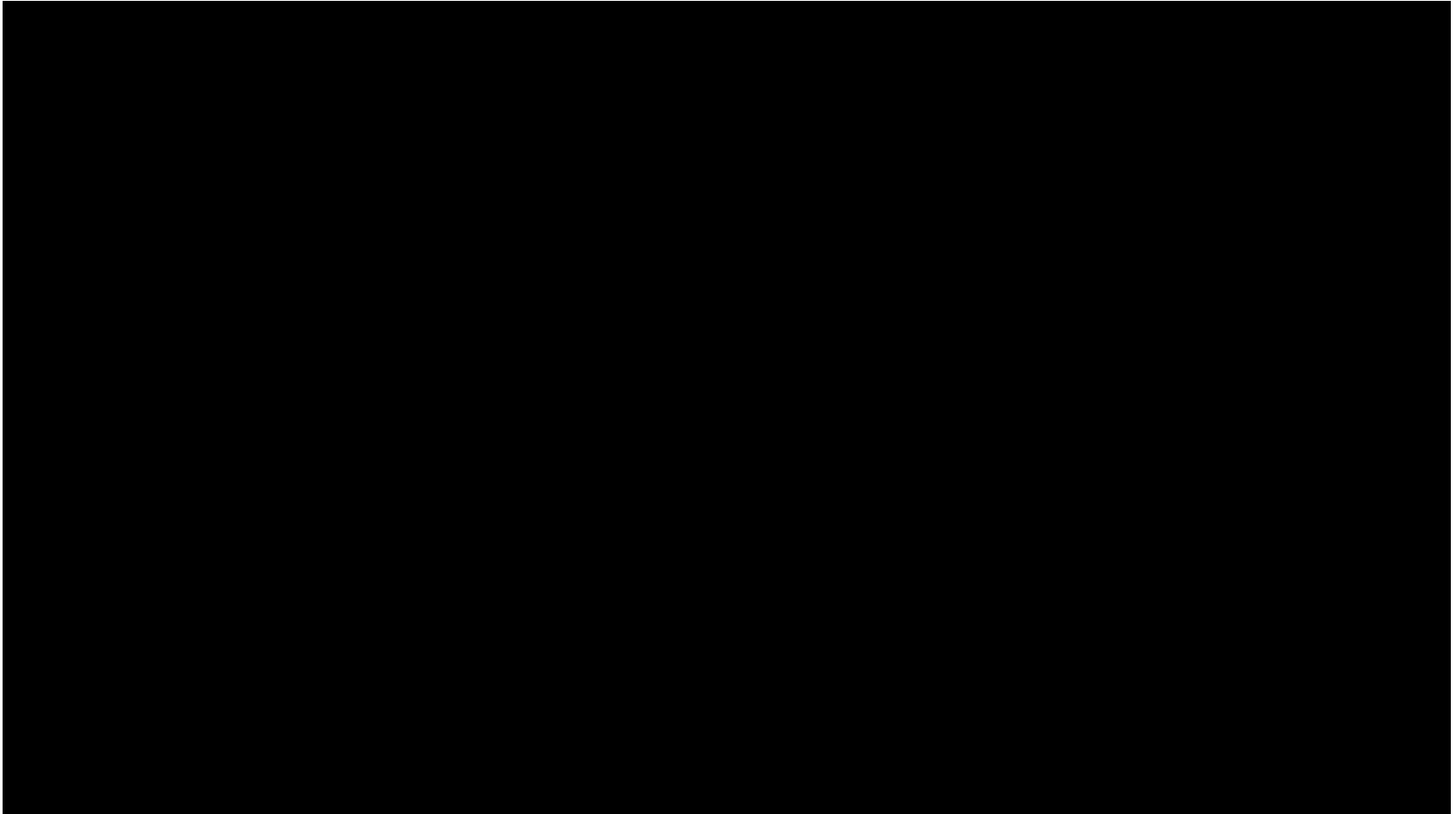


HumanoidVLM: Vision–Language–Guided Impedance Control for Contact-Rich Humanoid Manipulation

Yara Mahmoud, Yasheerah Yaqoot, Miguel Altamirano Cabrera, and Dzmitry Tsetserukou

Intelligent Space Robotics Laboratory, Center for Engineering Systems and Sciences,
Skolkovo Institute of Science and Technology, Moscow, Russia

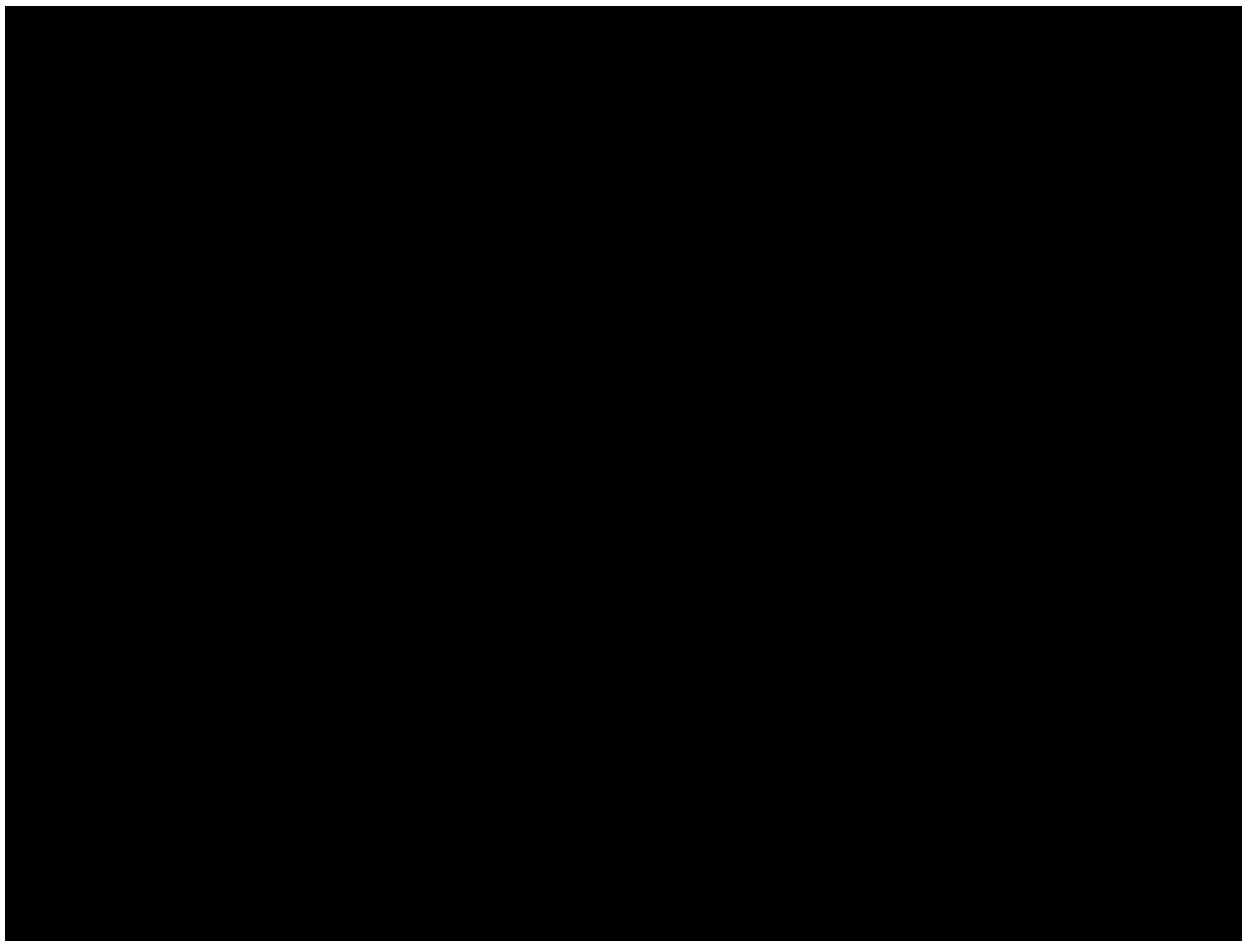
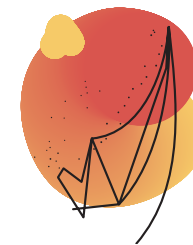
Motivacija

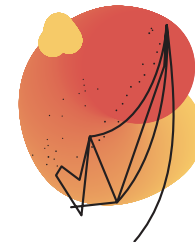


Motivacija



Motivacija





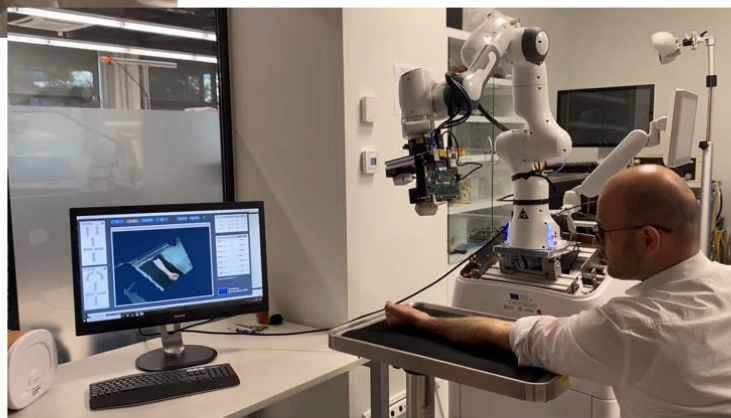
REMDOC - experiment

Remote planning



Financira
Europska unija
NextGenerationEU

Ultrasound scanning



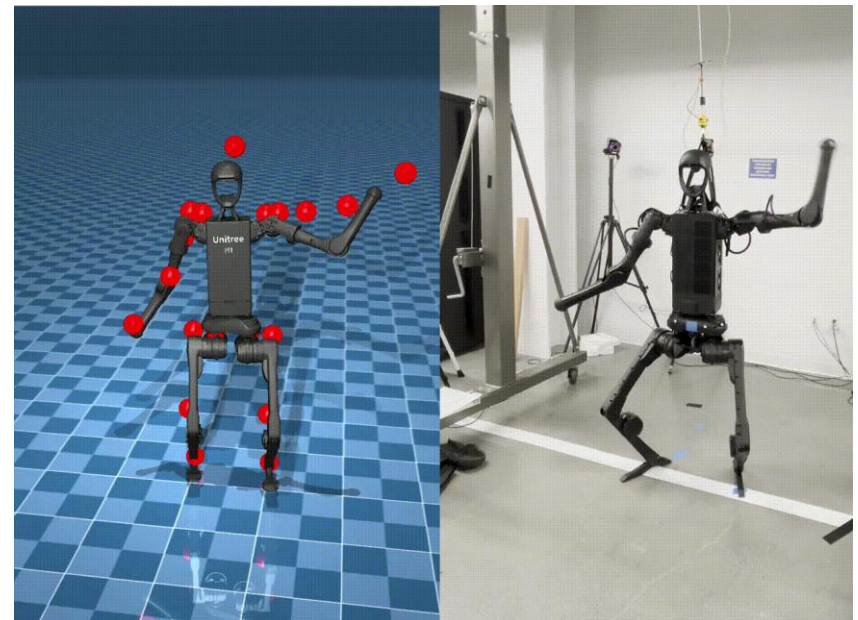
Upravljanje kompleksnim robotskim sustavima u MuJoCo simulatoru

OD KLASIČNOG PREMA OPTIMALNOM UPRAVLJANJU
ROBOTSКИH SUSTAVA

Kompleksni robotski sustavi u MuJoCo simulatoru

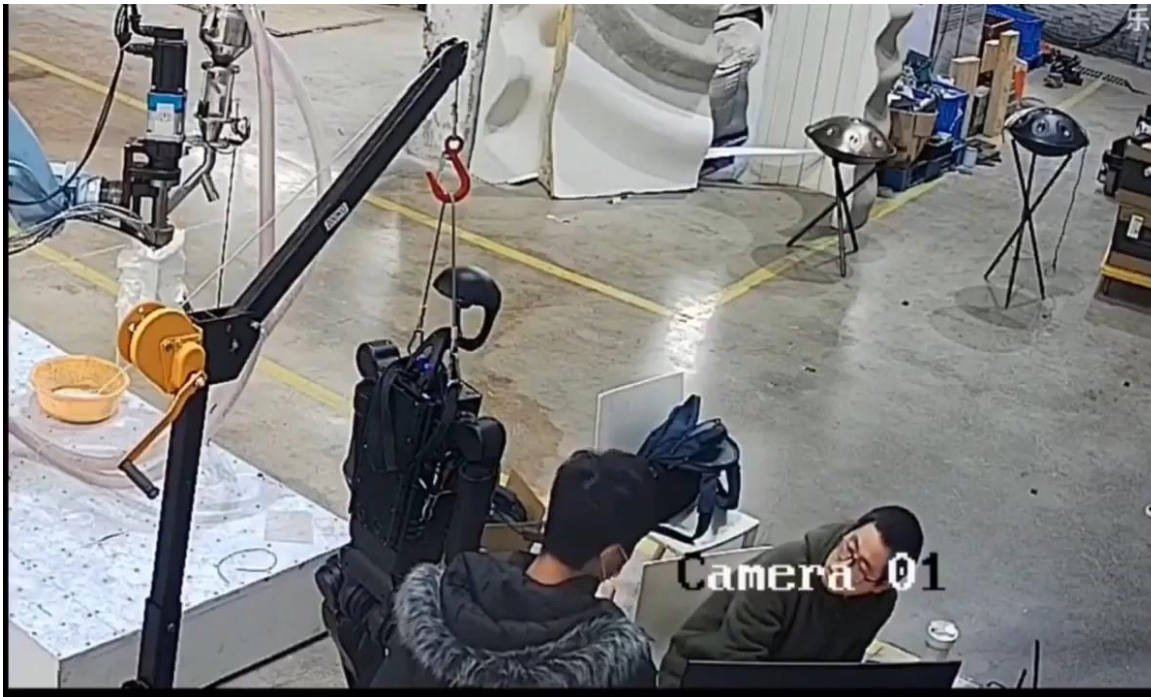
- U stvarnim robotskim sustavima pojavljuju se:
 - Veliki broj stupnjeva slobode
 - Nelinearna dinamika
 - Kontakt s okolinom
 - Coupling između dijelova sustava

UPRAVLJANJE VIŠE NIJE SAMO SLANJE
MOMENTA NA JEDAN ZGLOB – CIJELI
SUSTAV POSTAJE MEĐUSOBNO POVEZAN I
TE ODOSE JE POTREBNO RAZMATRATI



Kompleksni sustavi teško se testiraju direktno na hardveru

- Razvoj upravljanja na stvarnom robotu je:
 - Skup
 - Spor
 - Potencijalno opasan
- Greške u upravljanju mogu dovesti do:
 - Sudara robota s okolinom
 - Oštećenja aktuatora i senzora
 - Nestabilnog ponašanja sustava



KOD HUMANOIDNIH
ROBOTA DODATNI PROBLEM
PREDSTAVLJA ODRŽAVANJE
BALANSA, GDJE I MALE
GREŠKE MOGU UZROKOVATI
PAD SUSTAVA

MuJoCo Menagerie

- Biblioteka gotovih robotskih modela (kinematičkih i dinamičkih)
- Razvijena za MuJoCo simulator
- https://github.com/google-deepmind/mujoco_menagerie
- Sadrži:
 - Manipulatore (KUKA LBR iiwa 1, **Franka Emika Panda**)
 - Humanoide (Apptronik Apollo, TALOS, Unitree H1)
 - Četveronožne robote (Boston Dynamics Spot, Google Barkour)
 - Dronove (Bitcraze Crazyflie 2, Skydio X2)
- Modeli uključuju:
 - Geometriju
 - Mase i inercije
 - Zglobove i aktuaciju,
 - Kontaktne parametre

MJCF/XML zapis robota

PONAVLJANJE

- Robot je definiran pomoću MJCF XML zapisa
- XML zapis definira:
 - Kako su segmenti robota povezani
 - Koji tipovi zglobova postoje
 - Kako izgledaju pojedini dijelovi robota
 - Na koji način djeluju aktuatori
- Na temelju XML modela simulator računa:
 - Gibanje robota
 - Djelovanje sila i momenata
 - Kontaktne interakcije
 - Odziv sustava na upravljačke signale

Aktuatori i indeksi u MuJoCo-u

HUMANOIDNI ROBOT – DEMO2

- Kod kompleksnih robotskih sustava indeksi koji mogu, ali ne moraju biti isti:
 - Joint index → zglobovi definiraju kinematiku
 - DOF index → DOF-ovi definiraju dinamiku
 - Actuator index → aktuatori definiraju upravljanje
- Humanoidni sustavi imaju velik broj aktuatora
- Upravljanje često želimo provoditi samo nad dijelom sustava pa je potrebno pravilno mapirati: stanje sustava, akciju, dinamiku

```
Broj aktuatora (nu): 19

ACTUATOR 0
  actuator name : left_hip_yaw
  joint name    : left_hip_yaw
  dof index     : 6
  qpos index    : 7
  ctrl range    : [-200. 200.]

ACTUATOR 1
  actuator name : left_hip_roll
  joint name    : left_hip_roll
  dof index     : 7
  qpos index    : 8
  ctrl range    : [-200. 200.]
```

01_actuators.py

Vrste aktuatora u MuJoCo-u

- Vrijednosti koje šaljemo u 'dana.ctrl[i]' ne predstavljaju uvijek istu fizičku veličinu
- Značenje upravljačkog signala ovisi o tipu aktuatora definiranom u XML modelu robota

1. Position aktuator

```
<position joint="joint1" kp="100"/>
```

- Aktuator prima željenu poziciju zgloba
- MuJoCo interno generira moment koji pokušava dovesti zglob na zadanu poziciju
- Aktuator se ponaša poput internog proporcionalnog regulatora

$$\tau = k_p(q_d - q)$$
$$u = q_d$$

Vrste aktuatora u MuJoCo-u

2. Velocity aktuator

```
<velocity joint="joint1" kv="10"/>
```

- Aktuator prima željenu brzinu zgloba
- MuJoCo generira moment koji pokušava ostvariti zadanu brzinu gibanja

$$\tau = k_v(\dot{q}_d - \dot{q}), u = \dot{q}_d$$

3. Motor actuator (torque control)

```
<motor joint="joint1"/>
```

- Upravljački signal direktno predstavlja moment ili silu aktuatora
- MuJoCo tada:
 - Ne implementira interni regulator
 - Direktno koristi zadani moment u dinamici sustava

$$\tau = u$$

Hijerarhija upravljanja

1. Vanjski regulator

- Mi implementiramo (trajectory tracking, PID, impedance, LQR..)
- Npr. generiramo q_d

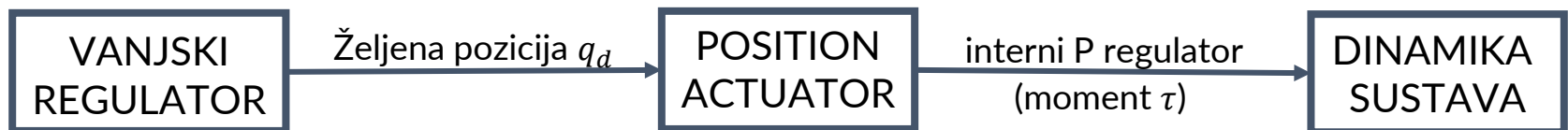
2. Aktuator

- Interpretira signal (pozicija/brzina/torque)
- Npr. za position aktuator: $\tau = k_p(q_d - q) = u$

BITNO!!!

Možemo li onda imati PD/PID control?

DA! – izlaz naših upravljačkih zakona ne smije biti direktni torque, nego npr. q_d



Primjeri kompleksnih robotskih sustava

HUMANOIDNI ROBOT

- **Unitree H1:**
 - Humanoidni robot
 - **Whole-body sustav** - problem održavanja balansa
- Pomicanje ruke ili trupa utječe na:
 - Raspodjelu masa
 - Kontaktne sile
 - Stabilnost sustava



Primjeri kompleksnih robotskih sustava

HUMANOIDNI ROBOT

- **Unitree H1:**
 - Radi izravno s torque naredbama

```
<actuator>
<motor class="h1" name="left_hip_yaw" joint="left_hip_yaw" ctrlrange="-200 200"/>
<motor class="h1" name="left_hip_roll" joint="left_hip_roll" ctrlrange="-200 200"/>
<motor class="h1" name="left_hip_pitch" joint="left_hip_pitch" ctrlrange="-200 200"/>
<motor class="h1" name="left_knee" joint="left_knee" ctrlrange="-300 300"/>
<motor class="h1" name="left_ankle" joint="left_ankle" ctrlrange="-40 40"/>
<motor class="h1" name="right_hip_yaw" joint="right_hip_yaw" ctrlrange="-200 200"/>
<motor class="h1" name="right_hip_roll" joint="right_hip_roll" ctrlrange="-200 200"/>
<motor class="h1" name="right_hip_pitch" joint="right_hip_pitch" ctrlrange="-200 200"/>
<motor class="h1" name="right_knee" joint="right_knee" ctrlrange="-300 300"/>
<motor class="h1" name="right_ankle" joint="right_ankle" ctrlrange="-40 40"/>
<motor class="h1" name="torso" joint="torso" ctrlrange="-200 200"/>
<motor class="h1" name="left_shoulder_pitch" joint="left_shoulder_pitch" ctrlrange="-40 40"/>
<motor class="h1" name="left_shoulder_roll" joint="left_shoulder_roll" ctrlrange="-40 40"/>
<motor class="h1" name="left_shoulder_yaw" joint="left_shoulder_yaw" ctrlrange="-18 18"/>
<motor class="h1" name="left_elbow" joint="left_elbow" ctrlrange="-18 18"/>
<motor class="h1" name="right_shoulder_pitch" joint="right_shoulder_pitch" ctrlrange="-40 40"/>
<motor class="h1" name="right_shoulder_roll" joint="right_shoulder_roll" ctrlrange="-40 40"/>
<motor class="h1" name="right_shoulder_yaw" joint="right_shoulder_yaw" ctrlrange="-18 18"/>
<motor class="h1" name="right_elbow" joint="right_elbow" ctrlrange="-18 18"/>
</actuator>
```

Primjeri kompleksnih robotskih sustava

HUMANOIDNI ROBOT – DEMO1 - Humanoid bez upravljanja

- U ovom primjeru svi upravljački signali aktuatora postavljeni su na 0
- Robot nema aktivno upravljanje $u = 0$
- **Kakvo ponašanje robota očekujemo**, ako je početna konfiguracija robota takva da stoji i **zašto?**
- Sustav reagira isključivo na gravitaciju, kontaktne sile i vlastitu dinamiku
- Robot nema:
 - Stabilizaciju
 - Kontrolu centra mase
 - Održavanje ravnoteže
- Bez aktivnog upravljanja:
 - Centar mase izlazi iz područja oslonca
 - Dolazi do gubitka ravnoteže
 - Sustav postaje nestabilan i robot pada

HUMANOIDNI ROBOTI ZAHTIJEVAJU KONTINUIRANU KOREKCIJU POLOŽAJA I RASPODJELE SILA KAKO BI ODRŽALI DINAMIČKU RAVNOTEŽU.

02_demo_Unitree_H1.py

Primjeri kompleksnih robotskih sustava

HUMANOIDNI ROBOT – DEMO2 - Humanoid s upravljanjem rukom

- Cilj je upravljati samo desnom rukom humanoidnog robota koristeći:
 - Joint-space reference (polinom 5. reda)
 - PD torque upravljanje
- Humanoid koristi motor aktuatora: $u = \tau$
- Upravljački zakon: $\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q})$
- Regulator:
 - Uspoređuje željenu i trenutnu konfiguraciju
 - Generira moment aktuatora
 - Prigušuje gibanje preko derivativnog člana

KOD HUMANOIDNOG ROBOTA UNITREE H1 AKTUATORI SU DEFINIRANI KAO MOTOR AKTUATORI, PA UPRAVLJAČKI SIGNAL PREDSTAVLJA DIREKTNI MOMENT AKTUATORA.

Primjeri kompleksnih robotskih sustava

PRIJELAZ: humanoid → manipulator

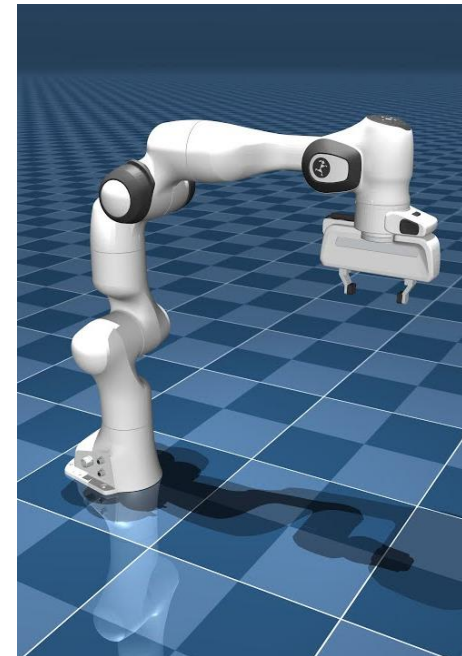
- Cilj trenutno nije rješavanje problema balansiranja humanoida
- Želimo se fokusirati na:
 - Generiranje trajektorija
 - Task-space upravljanje
 - Jacobian mapiranje
 - Impedance control
- Kod humanoida bi problem stabilizacije dominirao nad samim upravljanjem trajektorije
- **Koja vrsta robota bi nam omogućila fokus na problemima koji nas zanimaju?**

MANIPULATOR NAM OMOGUĆUJE DA PROUČAVAMO
NAPREDNE UPRAVLJAČKE ZAKONE BEZ DODATNOG
PROBLEMA ODRŽAVANJA DINAMIČKE RAVNOTEŽE.

Primjeri kompleksnih robotskih sustava

MANIPULATOR

- **Franka Emika Panda:**
 - 7 DOF redundantni manipulator
 - Fiksirana baza
 - Pogodan za task-space upravljanje
- Zbog fiksirane baze ne postoji problem održavanja balansa, pa se možemo fokusirati na:
 - Generiranje trajektorije
 - Task-space control
 - Jacobian i redundantnost sustava



Primjeri kompleksnih robotskih sustava

MANIPULATOR

- **Franka Emika Panda – tip aktuatora**

```
<general class="panda" name="actuator1" joint="joint1" gainprm="4500" biasprm="0 -4500 -450"/>  
<general class="panda" name="actuator2" joint="joint2" gainprm="4500" biasprm="0 -4500 -450" ctrlrange="-1.7628 1.7628"/>  
<general class="panda" name="actuator3" joint="joint3" gainprm="3500" biasprm="0 -3500 -350"/>  
<general class="panda" name="actuator4" joint="joint4" gainprm="3500" biasprm="0 -3500 -350" ctrlrange="-3.0718 -0.0698"/>  
<general class="panda" name="actuator5" joint="joint5" gainprm="2000" biasprm="0 -2000 -200" forcerange="-12 12"/>  
<general class="panda" name="actuator6" joint="joint6" gainprm="2000" biasprm="0 -2000 -200" forcerange="-12 12" ctrlrange="-0.0175 3.7525"/>  
<general class="panda" name="actuator7" joint="joint7" gainprm="2000" biasprm="0 -2000 -200" forcerange="-12 12"/>
```

- `<general>` nije tip upravljanja, nego fleksibilni aktuator model
- **Kako prepoznamo tip aktuatora?**
- Tip aktuatora određujemo iz jednadžbe koju implementira
- Parametri 'gainprm' i 'biasprm' definiraju odnos između:

- upravljačkog signala
- stanja sustava
- generiranog momenta

- Općenito vrijedi: $\tau = \text{gainprm} * u + \text{biasprm}[0] + \text{biasprm}[1]q + \text{biasprm}[2]\dot{q}$

kako upravljački signal utječe na
generirani moment aktuatora

Konstantni bias član

Pojačanje P člana

Pojačanje D člana

Upravljački signal (data.ctrl)

pozicija zgloba

brzina zgloba

Primjeri kompleksnih robotskih sustava

MANIPULATOR

- Franka Emika Panda – joint 1 (ostali su analogni)

```
<general class="panda" name="actuator1" joint="joint1" gainprm="4500" biasprm="0 -4500 -450"/>
```

$$\tau = 4500u - 4500q - 450\dot{q}$$

$$\tau = 4500(u - q) - 450\dot{q}$$

Ako vrijedi: $u = q_d$

$$\tau = 4500(q_d - q) - 450\dot{q}$$

$$\tau = K_p(q_d - q) - K_d\dot{q}$$

Primjeri kompleksnih robotskih sustava

General actuator - općenito

- Općenito, general actuator može implementirati:
 - position servo
 - velocity servo
 - torque actuator
 - tendon actuator
 - custom dynamics...
- Torque actuator: gainprm = "1", biasprm = "0 0 0"

$$\tau = u$$

Primjeri kompleksnih robotskih sustava

MANIPULATOR – DEMO3 – Upravljački zakon u task space-u

- U prošlim predavanjima definirali smo:
 - Kako generirati trajektoriju
 - Kako definirati grešku
 - Kako koristiti Jacobian za prijelaz task $\rightarrow$ joint space
- Što želimo napraviti prilikom implementacije?
 - Učitati model robota iz MuJoCo Menagerie
 - Očitati trenutno stanje robota
 - Izračunati task-space grešku
 - Preko Jacobiana dobiti gibanje zglobova
 - Poslati komandu aktuatorima

CILJ VIŠE NIJE SAMO RAZUMJETI JEDNADŽBU, NEGO
VIDJETI KAKO SE ONA PRETVARA U SIMULACIJSKI KOD.

Primjeri kompleksnih robotskih sustava

MANIPULATOR – DEMO3 – Upravljački zakon u task space-u

- Bitna razlika: stanje vs akcija:
 - **qpos** → položaji zglobova
 - **qvel** → brzine zglobova
 - **ctrl** → komande aktuatorima
- Kod kompleksnijih modela prvo moramo provjeriti:
 - Koje zglobove želimo koristiti
 - Koji aktuatori djeluju na te zglobove
 - Što ctrl zapravo predstavlja
- Kod Panda modela iz Menagerie komande aktuatorima su **pozicijske reference**, a ne direktni momenti.

Priprema (instalacija)

- Potrebno je instalirati Mujoco Menagerie
 - **Savjet:** napravite direktorij 'mujoco' u C disku `C:\mujoco\`
 - **U terminalu:** pozicionirajte se u napravljen direktorij `cd C:\mujoco`
 - **Klonirajte repo:** `git clone https://github.com/google-deepmind/mujoco_menagerie.git`
 - Kasnije u kodu pazite da je dobro referenciran direktorij, npr. za učitavanje modela Unitree H1 robota:

```
model = mujoco.MjModel.from_xml_path(  
    r"C:\mujoco\mujoco_menagerie\unitree_h1\scene.xml"  
)
```
- Proučite XML datoteke za Unitree H1 i Franka Emika Panda robote te usporedite s jednostavnim modelom 3DOF koji je generiran na prošloj vježbi

Zadatak 1 - Task Space upravljanje — Franka Emika Panda

- Zadajemo željenu trajektoriju end-effektora kao kružnicu u YZ ravnini (task space), a referenca pozicije prati sljedeći zakon:

$$x_d(t) = x_0 + \begin{bmatrix} 0 \\ r(\cos(\omega t) - 1) \\ r \sin(\omega t) \end{bmatrix}, r = 0.08 \text{ m}, \omega = 3.5 \text{ rad/s}$$

- Zadajemo željenu trajektoriju end-effektora kao kružnicu u YZ ravnini (task space), a referenca brzine prati sljedeći zakon:

$$\dot{x}_d(t) = \begin{bmatrix} 0 \\ -r\omega \sin(\omega t) \\ r\omega \cos(\omega t) \end{bmatrix}$$

- Orijentacija end-effektora drži se **konstantnom**: $R_d = R_0 = \text{const.}, \omega_d = 0$
- Pogreška u prostoru zadatka:
 - Pozicijska pogreška: $e_{pos} = x_d - x$
 - Rotacijska pogreška: $e_r = \frac{1}{2} (R_d R^T - R R_d^T)^V$ - iz antisimetrične razlike rotacijskih matrica
 - Napomena: operator $(\cdot)^V$ izvlači vektor iz antisimetrične matrice

Zadatak 1 - Task Space upravljanje — Franka Emika Panda

- Upravljački zakon u task space-u:
 - Željene brzine u task spaceu (6D):

$$\begin{bmatrix} v_{cmd} \\ \omega_{cmd} \end{bmatrix} = \begin{bmatrix} \dot{x}_d + K_{pos} \cdot e_{pos} \\ \omega_d + 2K_{rot} \cdot e_{rot} \end{bmatrix}, \quad K_{pos} = 15, K_{rot} = 10$$

- Preslikavanje u prostor zglobova:

- Jacobian matrica $J \in \mathbb{R}^{6 \times 7}$ dijeli se na translacijski i rotacijski dio: $J = \begin{bmatrix} J_p \\ J_R \end{bmatrix}$
- Željene brzine zglobova dobivaju se **pseudoinverzom**

$$\dot{q}_{cmd} = J^\dagger \begin{bmatrix} v_{cmd} \\ \omega_{cmd} \end{bmatrix} \quad J^\dagger = J^T (JJ^T + \lambda^2 I)^{-1}, \quad \lambda = 0.03$$

- Integracija i pozicijska naredba

- Referentni kutovi zglobova se iz brzina dobivaju Eulerovom integracijom:

$$q_{ref}(t + \Delta t) = q_{ref}(t) + \dot{q}_{cmd} \cdot \Delta t$$

- Referentni kutovi zglobova šalju se kao **pozicijska naredba** PD aktuatorima: $u = q_{ref}$

Zadatak 2 – vizualizacija podataka

- Nakon provedene simulacije task space upravljanja, logirali ste signale u datoteku "data/sim_data_task_space.npz"
- Potrebno je napisati '.py' skriptu kojom ćete vizualizirati:
 1. Praćenje pozicije - q_{ref} vs q_{meas}
 2. Praćenje brzine - $\dot{q}_{ref}$ vs $\dot{q}_{meas}$
 3. Upravljački signal - $u(t)$

Upravljanje interakcijom i kontaktnim silama

FORCE AND INTERACTION CONTROL

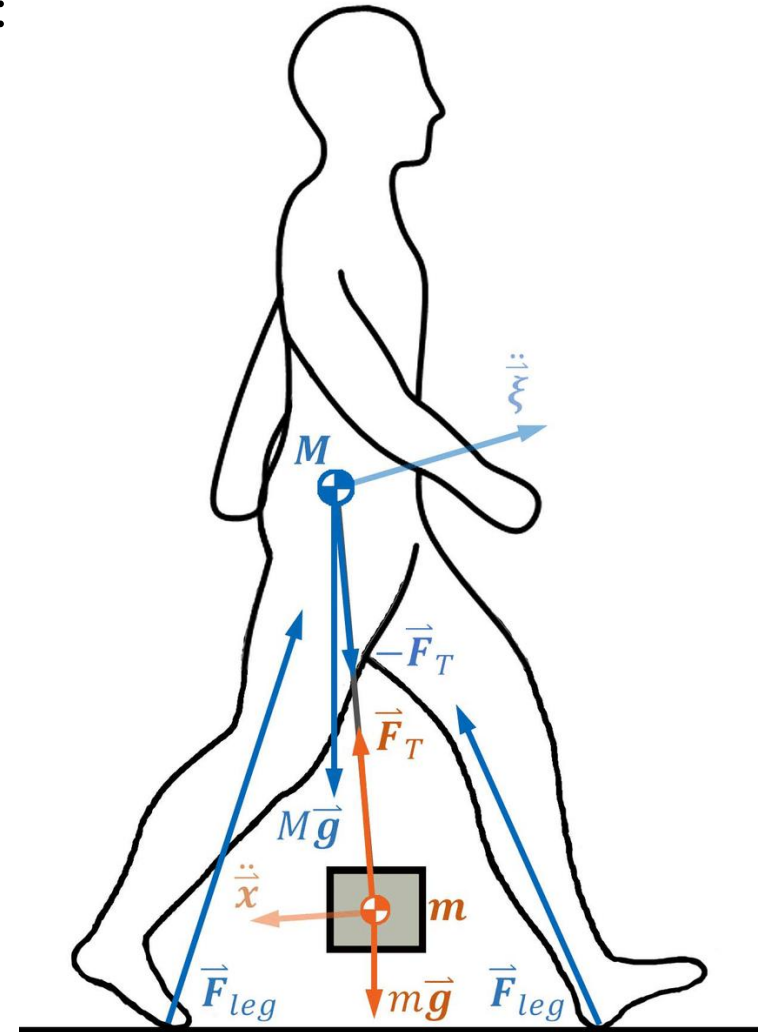
Zašto klasično pozicijsko upravljanje nije dovoljno?

- U stvarnim robotskim zadacima robot:
 - Dolazi u kontakt s okolinom
 - Djeluje silama na objekte
 - Prima sile iz okoline
- Primjeri:
 - Manipulacija objekata
 - Otvaranje vrata
 - Humanoidno hodanje
 - Kontakt s podlogom
 - Suradnja s čovjekom



Zašto klasično pozicijsko upravljanje nije dovoljno?

- U stvarnim robotskim zadacima robot:
 - Dolazi u kontakt s okolinom
 - Djeluje silama na objekte
 - Prima sile iz okoline
- Primjeri:
 - Manipulacija objekata
 - Otvaranje vrata
 - Humanoidno hodanje
 - Kontakt s podlogom
 - Suradnja s čovjekom



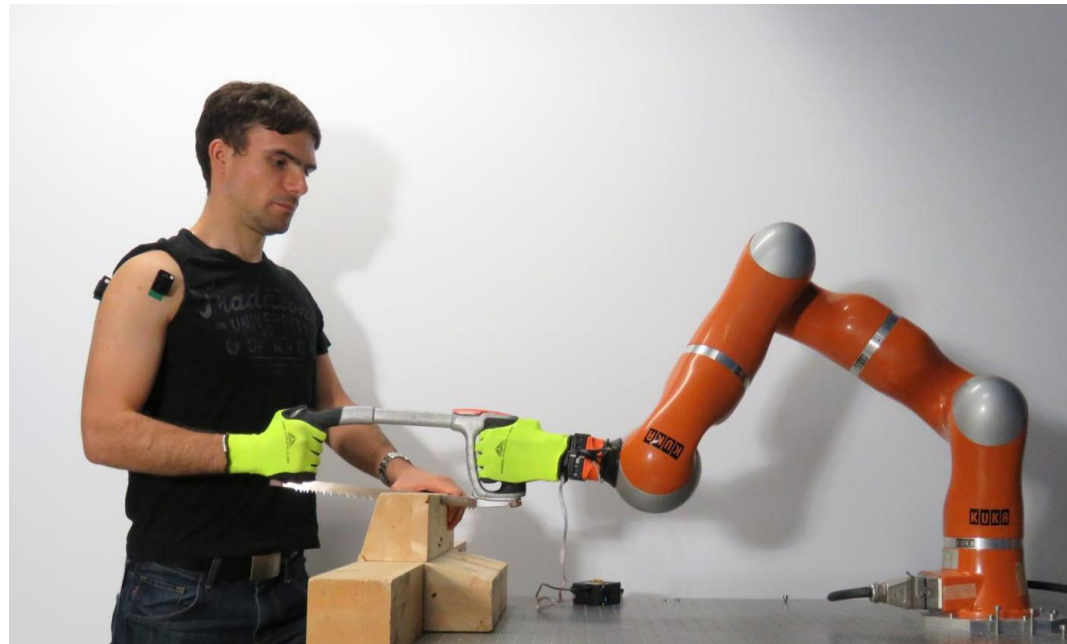
Zašto klasično pozicijsko upravljanje nije dovoljno?

- U stvarnim robotskim zadacima robot:

- Dolazi u kontakt s okolinom
- Djeluje silama na objekte
- Prima sile iz okoline

- Primjeri:

- Manipulacija objekata
- Otvaranje vrata
- Humanoidno hodanje
- Kontakt s podlogom
- Suradnja s čovjekom



Zašto klasično pozicijsko upravljanje nije dovoljno?

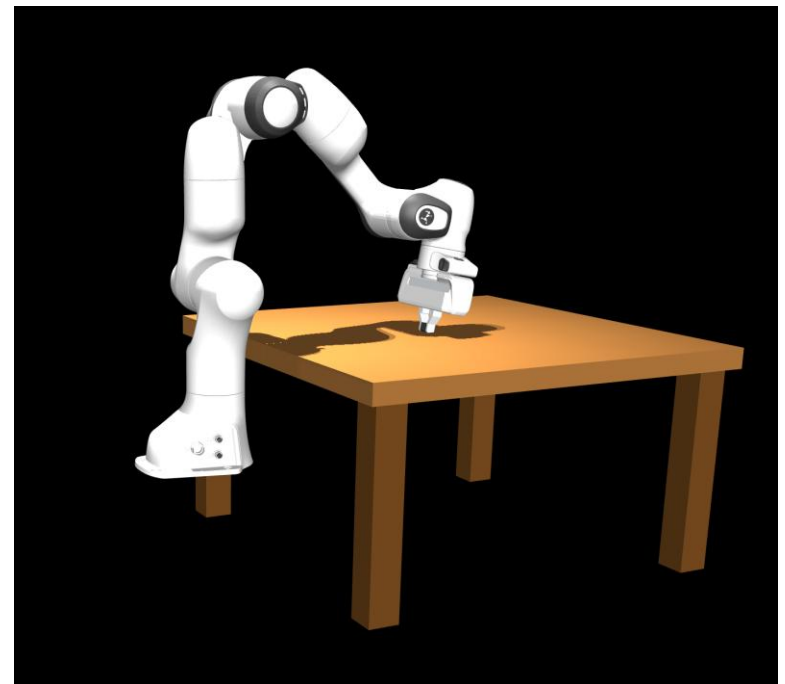
- U stvarnim robotskim zadacima robot:

- Dolazi u kontakt s okolinom
- Djeluje silama na objekte
- Prima sile iz okoline

KOD INTERAKCIJSKIH ZADATAKA
VAŽAN NIJE SAMO POLOŽAJ
ROBOTA, NEGO I NAČIN NA KOJI
ROBOT REAGIRA NA SILE IZ OKOLINE.

- Primjeri:

- Manipulacija objekata
- Otvaranje vrata
- Humanoidno hodanje
- **Kontakt s podlogom**
- Suradnja s čovjekom



Ideja compliant ponašanja

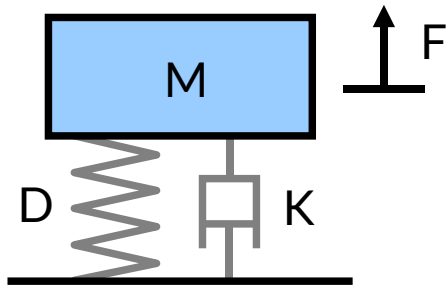
- Želimo da se robot ponaša:
 - Elastično
 - Prilagodljivo
 - “Meko” tijekom kontakta

COMPLIANCE CONTROL DEFINIRA KAKO
ROBOT REAGIRA NA KONTAKTNE SILE
TIJEKOM INTERAKCIJE S OKOLINOM.

- Intuicija:
 - Umjesto beskonačno krutog position controla želimo ponašanje slično:
 - Opruzi, prigušivaču, mehaničkom sustavu
- Posljedica je da robot može:
 - Apsorbirati kontaktne sile
 - Sigurnije interagirati s okolinom
 - Stabilnije izvršavati kontaktne zadatke

Impedance control

- Impedance control definira **odnos između gibanja robota i kontaktnih sila**
- Željeno ponašanje robota je poput sustava mase, opruge i prigušivača



$$F = M\ddot{x} + D\dot{x} + Kx$$

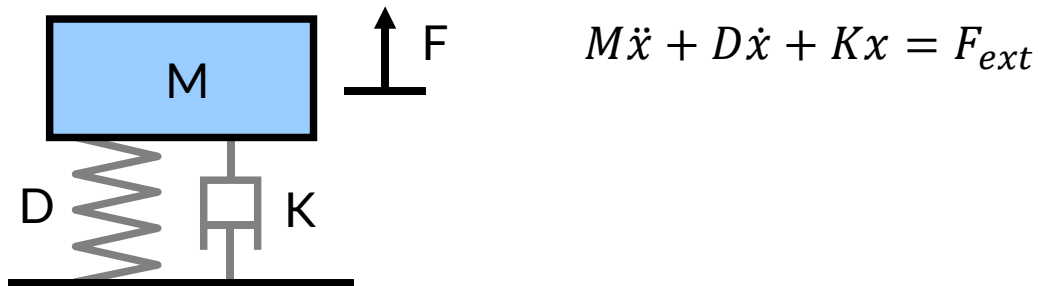
- $M \rightarrow$ virtualna masa (inercija) – uvijek uz akceleraciju
- $D \rightarrow$ prigušenje
- $K \rightarrow$ krutost sustava

- Intuicija je da, ako vanjska sila djeluje na robota:
 - Robot dopušta određeno odstupanje
 - Robot generira silu koja se suprotstavlja poremećaju

IMPEDANCE CONTROL DEFINIRA KAKO
GIBANJE ROBOTA GENERIRA KONTAKTNE SILE

Admittance control

- Admittance control definira **kako vanjske sile uzrokuju gibanje robota**



- Intuicija je da vanjska sila postaje ulaz sustava, a regulator nameće željeno gibanje robota kako bi ju omogućio

ADMITTANCE CONTROL DEFINIRA KAKO
KONTAKTNE SILE UZROKUJU GIBANJE ROBOTA.

Impedance vs. admittance control

	Impedance	Admittance
Zakon:	Gibanje $\rightarrow$ sila	Sila $\rightarrow$ gibanje
Ulaz:	Gibanje	Sila
Izlaz:	Sila	Gibanje
Kontroler:	Pogodno za torque control	Pogodno za position control
Ponašanje robota:	Robot se ponaša kao mehanička impedancija	Robot se ponaša kao mehanička admitancija

Zadatak 3 - Impedance upravljanje — Franka Emika Panda

- Motivacija i osnovna ideja:
 - Impedance upravljanje definira **mehaničko ponašanje** end-effektora umjesto da zadaje poziciju. Robot se ponaša kao virtualni sustav masa-prigušivač-opruga:

$$M_d \ddot{e} + D_d \dot{e} + K_d e = F_{ext}$$

- $M_d \rightarrow$ željena inercijska matrica $[kg]$
- $D_d \rightarrow$ matrica prigušenja $[Ns/m]$
- $K_d \rightarrow$ matrica krutosti $[N/m]$
- $F_{ext} \rightarrow$ vanjska sila $[N]$
- $e = x_d - x \rightarrow$ pozicijska greška $[m]$

- Željena inercijska matrica definira način na koji želimo da se EEF ponaša
- Npr. $M_d = \text{diag}(0.5, 0.5, 0.5)$ - Ponašaj se kao da end-effektor ima masu 0.5 kg u svakom smjeru
- Veliki $M_d \rightarrow$ end-effektor se "osjeća teško" $\rightarrow$ sporo reagira na sile $\rightarrow$ mekše, tromost
- Mali $M_d \rightarrow$ brzo reagira na sile $\rightarrow$ agresivniji odgovor

KLJUČNA RAZLIKA OD TASK SPACE UPRAVLJANJA: ROBOT **NE POKUŠAVA ELIMINIRATI**
POGREŠKU PO SVAKU CIJENU — DOPUŠTA POMAK PROPORCIONALAN PRIMIJENJENOJ SILI.

Zadatak 3 - Impedance upravljanje — Franka Emika Panda

- Definiranje proizvoljne scene kao XML datoteke

```
<mujoco model="panda_with_table">
  <include file="C:/m/mujoco_menagerie/franka_emika_panda/panda.xml"/>
  <visual>
    <headlight ambient="0.4 0.4 0.4" diffuse="0.8 0.8 0.8"/>
  </visual>
  <worldbody>
    <light pos="0 0 2.5" dir="0 0 -1" diffuse="0.8 0.8 0.8"/>
    <!-- stol -->
    <body name="table" pos="0.5 0 0.2">
      <geom name="table_top" type="box" size="0.4 0.4 0.02" rgba="0.6 0.4 0.2 1" friction="1 0.005 0.0001" contype="1" conaffinity="1"/>
      <geom name="leg1" type="box" size="0.03 0.03 0.2" pos=" 0.35 0.35 -0.22" rgba="0.4 0.25 0.1 1"/>
      <geom name="leg2" type="box" size="0.03 0.03 0.2" pos="-0.35 0.35 -0.22" rgba="0.4 0.25 0.1 1"/>
      <geom name="leg3" type="box" size="0.03 0.03 0.2" pos=" 0.35 -0.35 -0.22" rgba="0.4 0.25 0.1 1"/>
      <geom name="leg4" type="box" size="0.03 0.03 0.2" pos="-0.35 -0.35 -0.22" rgba="0.4 0.25 0.1 1"/>
    </body>
  </worldbody>
</mujoco>
```

- Umjesto da pišemo cijeli robot od nule, koristimo include koji učitava gotov Panda model iz MuJoCo Menagerie repozitorija
- Sve što dodajemo u <worldbody> postoji uz robota u istoj sceni
- Stol je definiran kao jedan <body> s pet geometrija — gornja ploča i četiri noge

Zadatak 3 - Impedance upravljanje — Franka Emika Panda

- Referenca i scena:
 - Robot se spušta prema stolu po **minimum jerk trajektoriji** duž Z osi

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = \frac{t}{T}$$

$$x_d(t) = x_0 + (x_{target} - x_0) \cdot s(T), \quad T = 6 \text{ sekundi}, \quad x_{target} = 0.22m$$

- x_{target} definiran je prema visini gornje plohe stola
 - Orijentacija EEF drži se konstantnom - $R_d = R_0 = \text{const.}$
- Upravljački zakon:

- Željeno ubrzanje u task spaceu dobiva se iz impedance zakona

$$\ddot{x}_{cmd} = \ddot{x}_d + M_d^{-1}(D_d\dot{e} + K_de + F_{ext})$$

Predznak sile ovisi o definiciji kontakta i smjeru osi.

- Parametri iznose:

- $M_d = \text{diag}(0.5, 0.5, 0.5)[kg]$
 - $D_d = \text{diag}(80, 80, 80)[Ns/m]$
 - $K_d = \text{diag}(400, 400, 400)[N/m]$

- Vanjska sila čista se direktno iz simulatora kao: $F_{ext} = \text{data.cfrc_ext}[\text{hand}][3:6]$

Zadatak 3 - Impedance upravljanje — Franka Emika Panda

- Preslikavanje u prostor zglobova:
 - Translacijski Jacobian $J_p \in \mathbb{R}^{3 \times 7}$ preslikava task space ubrzanje u zglobne momente:
$$\tau_{\text{task}} = M(q) \cdot J_p^\dagger \cdot \ddot{x}_{\text{cmd}}, \quad J^\dagger = J^T (JJ^T + \lambda^2 I)^{-1}, \quad \lambda = 0.01$$
 - Napomena, u ovom slučaju $M(q) \in \mathbb{R}^{7 \times 7}$ je masa inercije robota u joint space-u
- Null-space projektor:
 - Radimo s redundantnom kinematičkom strukturom – možemo null-space koristiti za regularizaciju joint space-a bez utjecaja na task space $N = I - J_p^T (J_p^T)^+$
$$\tau_{\text{rot}} = N(K_{p,\text{rot}}(q_{\text{init}} - q) - K_{d,\text{rot}}\dot{q})$$
$$K_{p,\text{rot}} = 300, \quad K_{d,\text{rot}} = 20$$
- Ukupna naredba momenta s gravitacijskom kompenzacijom
$$\tau_{\text{cmd}} = \tau_{\text{task}} + \tau_{\text{rot}} + \tau_{\text{grav}}$$
- Moment se šalje **direktno u zglobove** zaobilazeći position-controlled aktuator:
$$\text{qfr_applied} = \tau_{\text{cmd}}$$

Zadatak 3 - Impedance upravljanje — Franka Emika Panda

- Procjena kontaktne sile:
 - Mjeriti možemo samo vanjske sile, a kontaktne sile u task space-u koje nastaju djelovanjem robota možemo procijeniti preko momenata u joint space-u:

$$\hat{F}_c = (J_p^T)^+ (\tau_{cmd} - \tau_{grav})$$

- Ovo je ekvivalent virtualnog F/T senzora na wristu — vidljivo je u logiranim podacima kao F_{est}

Zadatak 4 - Vizualizacija i analiza

- Nakon provedene simulacije task space upravljanja, logirali ste signale u datoteku "data/sim_data_impedance.npz"
- Potrebno je napisati '.py' skriptu kojom ćete vizualizirati:
 1. Praćenje pozicije - q_{ref} vs q_{meas}
 2. Sila eksterna - $F_{ext}(t)$
 3. Procijenjena sila iz momenta- $F_{est}(t)$
 4. Momenti u joint space-u za sve zglobove